Projeto 4: Tema Livre

> Material para os alunos · React Native (Expo SDK 54) + Firebase · derivado do projeto-base de cadastro de alunos

1. Ideia do tema livre

Aqui **você escolhe o assunto**. Pode ser um app útil para o seu dia a dia, para um pequeno negócio, para a escola ou para a família. O que não é livre são os **requisitos técnicos**: o app precisa usar o que aprendemos com o projeto-base (login, Firestore, hooks, abas) e cumprir o mínimo descrito abaixo.

> **Antes de começar**, entregue a proposta (Atividade 1) e espere a aprovação do professor.

2. Requisitos mínimos obrigatórios

- Login e logout com Firebase Authentication.
- **Pelo menos 2 coleções** no Firestore (ou 1 coleção + 1 subcoleção) com relação clara entre elas.
- **CRUD completo** (cadastrar, listar, alterar e excluir) em pelo menos uma coleção.
- Um **hook próprio** para consultar o Firestore, seguindo o modelo `use-alunos.ts`.
- Mínimo de **5 telas** (contando o login) com navegação por abas.
- Validação dos campos e mensagens de erro/sucesso com `Toast`.
- **Regras de segurança** do Firestore que exijam usuário autenticado.
- Identidade visual própria e **sem nenhuma tela de exemplo** do template.
- Código em repositório Git, com `README.md`.

3. O que não pode

- Repetir o próprio projeto-base (cadastro de alunos) ou os temas 1, 2 e 3 só com outro nome.
- Guardar dados sensíveis reais de pessoas (CPF, endereço, saúde). Use dados fictícios.
- Publicar chaves ou senhas pessoais em repositório público.

4. O que você vai reaproveitar do projeto-base

Arquivo	O que faz
`firebase.ts`	Inicializa o Firebase e exporta `auth` e `db`.
`components/frm-login.tsx`	Tela de login com `signInWithEmailAndPassword`.
`components/frm-cad-aluno.tsx`	Formulário que cadastra (`addDoc`) e altera (`updateDoc`) conforme o parâmetro `id`.
`components/lst-aluno.tsx`	Lista (`FlatList`) com botões Alterar e Excluir (`deleteDoc`).
`hooks/use-alunos.ts`	Hook que consulta a coleção (`getDocs`) e um documento (`getDoc`).
`hooks/use-focus-refresh.ts`	Recarrega os dados toda vez que a tela recebe foco.

Arquivo	O que faz
`app/(tabs)/_layout.tsx`	Barra de abas com as cores do tema.
`app/(tabs)/index.tsx`	Tela inicial com botão Sair (`signOut`).

5. Cardápio de diferenciais (escolha no mínimo 3)

Diferencial	O que precisa ter
Subcoleção relacionada	Registros filhos de outro (ex.: itens de um pedido, comentários de uma tarefa) em `colecao/{id}/sub` .
Dados por usuário	Cada usuário vê apenas os **seus** registros (campo `uid` gravado com `auth.currentUser.uid` + regra de segurança correspondente).
Busca e filtros	Busca por texto e filtro por categoria/status na lista, usando `where` ou filtro no app.
Ordenação e agrupamento	Lista ordenada (`orderBy`) e/ou agrupada por categoria, data ou status.
Painel de resumo	Tela inicial com contadores/totais calculados a partir do Firestore (ex.: itens por status, soma de valores).
Mudança de status com um toque	Alternar o estado de um registro (ex.: pendente ↔ concluído) direto na lista.
Cadastro de usuário	Tela de criar conta (`createUserWithEmailAndPassword`) e recuperação de senha.
Perfil do usuário	Tela com os dados do usuário logado e edição de um documento `perfis/{uid}` .
Recurso nativo do Expo	Usar um módulo do Expo SDK 54 (ex.: localização, câmera ou seletor de imagem) integrado ao app.
Atualização em tempo real	Usar `onSnapshot` para a lista atualizar sozinha, sem recarregar a tela.

6. Ideias de inspiração (opcional)

Se não souber por onde começar:

- Agenda de consultas ou compromissos
- Controle de gastos pessoais
- Lista de compras compartilhada
- Diário de estudos ou de treinos
- Biblioteca de livros, filmes ou jogos
- Livro de receitas
- Cadastro de pets para adoção
- Organizador de eventos e convidados
- Controle de tarefas de uma equipe
- Catálogo de produtos de um pequeno comércio

7. Modelo da proposta (1 página)

Item	O que escrever
Nome do app	Um nome curto e a ideia em uma frase
Problema	Que problema resolve e para quem
Funcionalidades	Lista das principais funcionalidades
Dados	Coleções e subcoleções, com os campos e tipos de cada uma
Telas	Lista das telas (mínimo 5) e como o usuário navega entre elas
Diferenciais	Os 3 (ou mais) itens do cardápio que pretende fazer
Integrantes	Nomes e quem faz o quê

8. Atividades

As atividades são sequenciais. Total: **100 pontos**.

Atividade 1 – Proposta e preparação do ambiente (5 pontos)

O que fazer

- Escrever a **proposta do projeto** (1 página, modelo na seção 7): problema, público, funcionalidades, dados e telas.
- Entregar a proposta ao professor e **aguardar a aprovação** antes de seguir para as demais atividades.
- Criar o **seu** projeto no console do Firebase, ativar **Authentication (e-mail/senha)** e **Firestore**.
- Trocar a configuração em `firebase.ts` e cadastrar um usuário de teste.
- Definir regras do Firestore que aceitem apenas usuários autenticados (`request.auth != null`).

Entregável: Proposta aprovada + print do app rodando e do console do Firebase.

Critério principal: Proposta completa e viável; Firebase é o do grupo; regras não estão abertas para qualquer pessoa.

Atividade 2 – Autenticação (login e logout) (10 pontos)

O que fazer

- Adaptar a tela de login do modelo à identidade do seu app.
- Mostrar mensagem de erro com `Toast` quando usuário/senha forem inválidos.
- Implementar o botão **Sair** que encerra a sessão e volta ao login (`router.replace('/(auth)')`).

Entregável: Vídeo curto (até 1 min) ou prints: login correto, login errado, logout.

Critério principal: Login/erro/logout funcionam; o Sair volta para a tela de login.

Atividade 3 – Cadastro da entidade principal (10 pontos)

O que fazer

- Criar o formulário com todos os campos definidos na proposta, validando obrigatórios e tipos.
- Gravar com `addDoc` e mostrar `Toast` de sucesso ou erro.
- Limpar o formulário após salvar.

Entregável: Prints do formulário preenchido e do documento criado no Firestore.

Critério principal: Dados chegam ao Firestore com os tipos certos (números como `number`).

Atividade 4 – Listagem (10 pontos)

O que fazer

- Criar um **hook próprio** (`hooks/use-<seu-tema>.ts`) seguindo o modelo `use-alunos.ts`.
- Listar com `FlatList` em cartões e recarregar ao voltar para a tela (`useFocusRefresh`).
- Mostrar mensagem amigável quando a lista estiver vazia.

Entregável: Print da lista com pelo menos 5 registros.

Critério principal: A lista atualiza sozinha após cadastrar/alterar/excluir.

Atividade 5 – Alteração e exclusão (15 pontos)

O que fazer

- Botão **Alterar** abre o formulário com os dados carregados (`getDoc` + parâmetro `id`).
- No modo edição, o botão muda para **Alterar** e grava com `updateDoc`.
- Botão **Excluir** com confirmação (`Alert`) antes de `deleteDoc`.

Entregável: Vídeo curto ou prints do fluxo: alterar um registro e excluir outro.

Critério principal: Edição não cria registro duplicado; exclusão pede confirmação.

Atividade 6 – Identidade visual (5 pontos)

O que fazer

- Definir (e justificar na documentação) a paleta de cores e o nome do app.
- Usar nomes de abas claros e ícones coerentes.
- Remover do app todo conteúdo de exemplo do template do Expo.

Entregável: Prints de todas as telas.

Critério principal: Visual consistente entre telas; nenhuma tela de template sobrando.

Atividade 7 – Diferenciais escolhidos (cardápio) (20 pontos)

O que fazer

- Escolher **no mínimo 3 itens** do cardápio de diferenciais e implementá-los (seção 5).
- Pelo menos **1 item** deve envolver uma **segunda coleção ou subcoleção**.
- Registrar na documentação quais itens foram escolhidos e por quê.

Entregável: Vídeo curto (até 2 min) mostrando cada diferencial e prints do Firestore.

Critério principal: Os itens funcionam e os dados ficam consistentes entre as coleções.

Atividade 8 – Documentação e repositório (10 pontos)

O que fazer

- Escrever o `README.md` do seu projeto: objetivo, como rodar, telas e modelo de dados.
- Comentar o código nos pontos principais (hooks e consultas ao Firestore).
- Subir o código em um repositório Git com commits ao longo do projeto.

Entregável: Link do repositório + README + apresentação.

Critério principal: Qualquer colega consegue rodar o projeto seguindo o seu README.

Atividade 9 – Apresentação em grupo (até 15 minutos) (15 pontos)

O que fazer

- Montar os slides seguindo o **roteiro de apresentação** descrito no fim deste documento.
- Mostrar o app funcionando (demonstração ao vivo ou vídeo de reserva).
- Explicar a ferramenta (Expo/React Native e Firebase) e os **desafios** que o grupo enfrentou com o framework e com a tecnologia.
- Dividir a fala entre **todos** os integrantes e respeitar o tempo de 15 minutos.

Entregável: Slides (PDF) entregues antes da apresentação + apresentação oral.

Critério principal: Cumpre o roteiro, o tempo e mostra domínio do que foi construído (cada integrante responde perguntas).

Desafios extras (opcionais, +5 pontos cada, até 10)

- Modo escuro respeitando o tema do sistema
- Publicar a versão web e apresentar o link

9. Cronograma sugerido

Antes da semana 1: entregar a proposta e aguardar a aprovação.

Período	Atividades
Semana 1	Atividades 1 e 2
Semana 2	Atividades 3 e 4
Semana 3	Atividades 5 e 6
Semana 4	Atividade 7
Semana 5	Atividades 8 e 9 (documentação e apresentações)

10. Avaliação

Atividade	Pontos
1 – Proposta e preparação do ambiente	5
2 – Autenticação (login e logout)	10
3 – Cadastro da entidade principal	10
4 – Listagem	10
5 – Alteração e exclusão	15
6 – Identidade visual	5
7 – Diferenciais escolhidos (cardápio)	20
8 – Documentação e repositório	10

Atividade	Pontos
9 – Apresentação em grupo (até 15 minutos)	15
Total	**100**

O que descontamos: projeto sem aprovação da proposta, requisitos mínimos faltando, chaves/senhas pessoais no repositório público, código copiado sem entender, telas do template sobrando, app que não abre.

11. Dicas importantes

- **Leia a documentação da versão certa**: este projeto usa Expo SDK 54 (`https://docs.expo.dev/versions/v54.0.0/`).
- Comece pequeno: faça o CRUD de uma coleção funcionando antes de pensar nos diferenciais.
- Faça commits pequenos e frequentes: um por atividade, no mínimo.
- Se o Firestore recusar a gravação, veja o código do erro no `Toast` e revise as regras de segurança.
- Paleta de cores do projeto-base (use ou troque): `#1a3a8f` azul principal, `#002d66` azul escuro (texto), `#F7941D` laranja de destaque (botões), `#f5f7fc` fundo claro, `#e85d04` laranja-escuro (ações destrutivas).

12. Apresentação final e roteiro (até 15 minutos por grupo)

Cada grupo apresentará o projeto para a turma. A apresentação vale **15 pontos** (Atividade 9). Todos os integrantes devem falar e responder perguntas.

Roteiro sugerido

#	Tópico	Tempo	O que abordar
1	Abertura e problema	1 min	Nome do grupo, tema escolhido e o problema real que o app resolve.
2	Demonstração do app	3 min	Fluxo completo: login, cadastro, lista, alteração, exclusão e a funcionalidade diferencial. Tenha um vídeo de reserva.
3	A ferramenta: Expo e React Native	2 min	O que é React Native e o que o Expo acrescenta (Expo Go, Expo Router, rodar no celular e no navegador). Por que usar.
4	Firebase: Authentication e Firestore	2 min	Como funciona o login por e-mail/senha, coleções e subcoleções, e as regras de segurança. Mostre o console.
5	Arquitetura do projeto	2 min	Organização de pastas, componentes, hooks (ex.: `use-alunos`) e o caminho de um dado: tela → hook → Firestore → tela.
6	Desafios com o framework	2 min	Problemas com React Native/Expo/Expo Router e como foram resolvidos (veja as sugestões do seu tema).
7	Desafios com a tecnologia	2 min	Problemas com Firebase/Firestore/TypeScript e como foram resolvidos (veja as sugestões do seu tema).
8	Aprendizados e próximos passos	1 min	O que o grupo aprendeu, o que faria diferente e o que melhoraria no app.

#	Tópico	Tempo	O que abordar
	Total	**15 min**	

Desafios sugeridos para o seu tema

Escolha os que o seu grupo realmente enfrentou e conte **o problema, o que tentaram e a solução**.

Framework (React Native / Expo / Expo Router)

- Decidir como dividir o app em telas e abas e navegar entre elas passando parâmetros (Expo Router)
- Manter as telas atualizadas ao voltar para elas sem causar loop de renderização

Tecnologia (Firebase / Firestore / TypeScript)

- Modelar os dados no Firestore (o que vira coleção, o que vira subcoleção)
- Escrever regras de segurança que combinem com o modelo escolhido

> Além dos sugeridos, vale citar os desafios comuns: rotas com o mesmo caminho (`/` do login e da tela inicial), loop de renderização por callback não memorizado e números gravados como texto.

Checklist antes de apresentar

- Slides prontos (cerca de 1 por minuto) e enviados em PDF.
- App rodando, com dados de exemplo já cadastrados.
- Vídeo de reserva caso a internet ou o Firebase falhe.
- Falas divididas entre todos os integrantes; tempo ensaiado.
- Chaves e senhas pessoais **fora** dos slides.

Avaliação da apresentação

Critério	Pontos
Conteúdo técnico (ferramenta e arquitetura corretas)	5
Desafios enfrentados e soluções (framework e tecnologia)	4
Demonstração funcionando	3
Clareza, divisão da fala e respeito ao tempo	3
Total	**15**

13. Referências

Expo

- Documentação do Expo SDK 54 (versão do projeto)
- Expo Router – introdução
- Expo Router – abas (Tabs)
- Usando Firebase com Expo

Firebase

- Firestore – visão geral
- Firestore – adicionar dados (addDoc, setDoc, updateDoc)
- Firestore – ler dados (getDoc, getDocs)
- Firestore – excluir dados (deleteDoc)
- Firestore – consultas simples e compostas (where, orderBy)
- Firestore – subcoleções e modelo de dados
- Firestore – regras de segurança
- Authentication – e-mail e senha na Web

React e React Native

- React – useState
- React – useCallback
- React – criando seus próprios Hooks
- React Native – FlatList
- React Native – TextInput
- React Native – StyleSheet
- react-native-toast-message

TypeScript

- TypeScript Handbook